

QUESTION 1

If `max` is an `int` which holds the maximum positive `int` value for a particular compiler, what is the value of `max + max`?

- A. 0 B. -1 C. -2 D. -max E. None of these

QUESTION 2

What is output by the code to the left if `input.txt` contains the data below?

2 4 6 8 10

- A. 10 8 6 4 2 B. 10108642
C. 108642 D. 0 1 8 6 4 2
E. None of these

```
ifstream in("input.txt");
apstack<int> stk;
int i;

while (in >> i) stk.push(i);

while (!stk.isEmpty()) {
    cout << stk.top();
    stk.pop();
}
```

QUESTION 3

If `apvector<int> v` holds the values {100, 25, 10, 5, 1}, what is output by the function call `change(189, v)`?

- A. 1 7 18 37 189 B. 189 37 18 7 1
C. 189 0 0 0 0 D. 1 3 1 0 4
E. None of these

```
void change(int amt,
            apvector<int> money) {
    int i=0;
    while (i<money.length()) {
        cout << amt/money[i] << ' ';
        amt%=money[i];
        ++i;
    }
}
```

QUESTION 4

If `apvector<int> v` holds the values {1, 5, 10, 25, 100}, what is output by the function call `change(189, v)`?

- A. 1 7 18 37 189 B. 189 37 18 7 1
C. 189 0 0 0 0 D. 1 3 1 0 4
E. None of these

QUESTION 5

What is returned by the function call `recursion(6,4)`?

- A. 0 B. 36 C. 1296
D. 24 E. None of these

```
int recursion(int x, int y) {
    if (y==0) return 1;
    else if (y%2==0)
        return recursion(x*x,y/2);
    else
        return x*recursion(x*x,y/2);
}
```

QUESTION 6

What is returned by the function call `recursion(2,7)`?

- A. 0 B. 14 C. 1048596
D. 128 E. None of these

QUESTION 7

Which of these declares an array of subscribers of length 10?

- A. `s=subscriber[10];` B. `subscriber[10] s;`
- C. `subscriber[] s(10);` D. `subscriber s[10];`
- E. None of these

```
class address {
    // code omitted
};

struct subscriber {
    apstring lastname;
    apstring firstname;
    address a;
    apvector<apstring>
        magazines(10);
};
```

QUESTION 8

Which of these checks to see whether `subscriber s1` and `s2` have the same `address`?

- A. `s1->address = s2->address`
- B. `s1.address = s2.address`
- C. `s1->a == s2->a`
- D. `s1.a == s2.a`
- E. None of these

QUESTION 9

The list of magazines a reader is subscribed to is always stored in the front of the vector `magazines`, with an empty string in the other spots. Which of these counts the number of magazines for `subscriber s`? Assume `int count` is initialized to 0.

- A.

```
for (int i=0; i<=10; ++i)
    if (s.magazines[i]!="") ++count;
```
- B.

```
for (; (count<10) &&
    (s.magazines[count]!="") &&
    ++count; );
```
- C.

```
while ((count<10) &&
    (s[magazines][count]!=""))
    ++count;
```
- D.

```
i=10;
while(i && s.magazines[i]=="") --i;
count=i;
```
- E. None of these

QUESTION 10

$12345_{16} + 67890_{16} =$

- A. 80235_{16} B. $79BD5_{16}$ C. $79EE5_{16}$ D. 80325_{16} E. None of these

QUESTION 11

Which of the following could be used in any C++ program to set the color of plant p to red?

- A. p.color=RED; B. p.color="red";
 C. p.set_color(RED); D. p.set_color("red");
 E. More than one of these

```
enum color {RED, ORANGE, YELLOW,
GREEN, BLUE, PURPLE, PINK,
WHITE, BLACK};
```

```
class plant {
// The class includes methods
// and data not listed.
```

```
public:
void set_color(color c);
```

```
private:
color c;
};
```

QUESTION 12

What is returned by the method set_color()?

- A. NULL B. nothing
 C. the color set D. the plant whose color is set
 E. Nothing

QUESTION 13

What is output by the code to the right on this input?

1 2 3 4 5 6 7 8 9 10

- A. 11 B. 45 C. 3628800
 D. 0 E. None of these

```
int i,j=0;
while (cin >> i) {
j^=i;
}
cout << j;
```

QUESTION 14

What is output by the function call output(3.14159, 0)?

- A. 3. B. 3.1
 C. 3.14 D. 0
 E. None of these

```
void output(double x, int y) {
cout << (int)x;
if (y>0) {
cout << '.';
x -= (int)x;
while (y-- > 0) {
x*=10;
cout << (int)x;
x -= (int)x;
}
}
}
```

QUESTION 15

What is output by the function call output(3.14159, 4)?

- A. 3.14159 B. 3.1415
 C. 3.141 D. 3.1416
 E. None of these

QUESTION 16

What correctly fills <*1> in the code to the right to test whether the character in position j of string v[i] is in the string choices?

- A. choices[0].find(v[i][j])!=1
- B. v[i][j].find(choices)==-1
- C. find(choices, v[i][j])==-1
- D. choices.find(v[i][j])==-1
- E. None of these

menu.h:

```
class menu {
public:
    menu(apvector<apstring> v);
    void display();
    int choice();
private:
    apvector<apstring> items;
    apstring choices;
};
```

menu.cpp

```
#include "menu.h"

menu::menu(apvector<apstring> v) {
    items = v;
    choices="";
    for (int i=0; i<v.length(); ++i)
    {
        for (int j=0; j<v[i].length(); ++j) {
            if (<*1>) {
                choices+=v[i][j];
                items[i]=v[i].substr(0,j)+
                    '*' +v[i][j]+'*' +
                    v[i].substr(j+1,
                        v[i].length()-j-1);
                break;
            }
        }
    }
}

void menu::display() {
    for (int i=0; i<items.length(); ++i)
        cout << items[i] << endl;
}

int menu::choice() {
    // reads a character from the
    // user and returns the number
    // of the corresponding menu
    // item
}
```

QUESTION 17

Assume that <*1> is filled in correctly. What is output by the code below?

```
apvector<apstring> v(4);
v[0] = "File";
v[1] = "Edit";
v[2] = "Format";
v[3] = "Help";
menu m(v);
m.display();
```

- A. *File*
Edit
Format
Help
- B. *F*File
*E*Edit
*0*Format
*H*Help
- C. *F*ile
*E*dit
*F*ormat
*H*elp
- D. *F*ile
*E*dit
F*o*rm
*H*elp
- E. None of these

QUESTION 18

Which of these functions from the math.h library is used to compute the base 10 logarithm of a number?

- A. log()
- B. ln()
- C. flog()
- D. logarithm()
- E. None of these

QUESTION 19

Which of these function calls returns the index of `apstring s` in `apstring A[]`?

- A. `find<apstring>(A[0],s)`
- B. `find(A,s)`
- C. `find(A[s])`
- D. `find<apstring>(A,s)`
- E. None of these

```
template <class itemType>
int find(itemType* array,
        itemType item) {
    int count=0;
    while (*array!=item) {
        ++count;
        ++array;
    }
    return count;
}
```

QUESTION 20

Which of these function calls returns the index of `int x` in `int A[]`?

- A. `find<int>(A[0],x)`
- B. `find(&(A[0]),x)`
- C. `find(A[x])`
- D. `find<int>(A,x)`
- E. None of these

QUESTION 21

What is returned by the function call `f(10)`?

- A. 10
- B. 100
- C. 5
- D. 50
- E. None of these

```
int f(int x) {
    int count = 0;
    for (int i=1; i<x; i*=2)
        for (int j=1; j<x; j*=2)
            ++count;
    return count;
}
```

QUESTION 22

What is the running time of `f(n)`? Choose the smallest correct answer.

- A. $O(\log n)$
- B. $O(n)$
- C. $O((\log n)^2)$
- D. $O(n^2)$
- E. None of these

QUESTION 23

Which of these expressions is represented by the truth table below?

a	b	c
0	0	0
0	1	1
1	0	1
1	1	0

- A. $c = a || b$
- B. $c = (a \& \& !b) || (b \& \& !a)$
- C. $c = !(a \& \& b) || !(a || b)$
- D. B and C
- E. None of these

QUESTION 24

What replaces **<*1>** and **<*2>** in the code to the right to access the **data** and **next** elements of **n**?

- A. `n->data` and `n->next`
- B. `n.data` and `n.next`
- C. `n(data)` and `n(next)`
- D. `n[data]` and `n[next]`
- E. None of these

For the remaining questions assume that **<*1>** and **<*2>** are filled in correctly.

QUESTION 25

Where are elements added by the method `add()`?

- A. the beginning of the list
- B. the end of the list
- C. into sorted order in the list
- D. into a random position in the list
- E. None of these

QUESTION 26

What happens to the memory used by a `list` created as a local variable in a function when the function ends?

- A. It is unavailable to be reallocated
- B. The memory used by the `list` object is freed for reallocation, but the memory used by the `nodes` it contains is not.
- C. The entire `list`, including its `nodes` are freed for reallocation
- D. It is garbage collected later in the program
- E. None of these

QUESTION 27

Which of these replaces **<*1>** in the code to the right so that `read_name()` reads a line of input from the input stream `in` to get a name?

- A. `in >> s;`
- B. `cin >> s;`
- C. `in.getline(s);`
- D. `getline(in,s);`
- E. None of these

```
list.h:
struct node {
    apstring data;
    node * next;
};

class list {
private:
    node * head;
public:
    list();
    void add(apstring s);
    void print();
};
```

```
list.cpp:
#include "list.h"

list::list() {
    head=NULL;
}

void list::add(apstring s) {
    node * n = new node;
    <*1>=s;
    <*2>=head;
    head=n;
}

void list::print() {
    node * n = head;
    while (n!=NULL) {
        cout << <*1>;
        n=<*2>;
    }
}
```

```
apstring read_name(ifstream in) {
    apstring s;
    <*1>
    return s;
}
```

QUESTION 28

When `apvector<int> v` below is sorted with the function call `sort(v)`, what will be the position of the element 17 after finishing the recursive call to `sort1()` marked with a comment in the code to the right?

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
0	17	3	5	14	9	12	7

- A. 1 B. 3 C. 5
D. 7 E. None of these

QUESTION 29

Which sorting algorithm is implemented by the code to the right?

- A. Selection sort B. Insertion sort
C. Quicksort D. Merge sort
E. None of these

QUESTION 30

What is the worst case running time of the `sort()` function on a vector of length n ? Choose the smallest correct answer.

- A. $O(n)$ B. $O(n^2)$ C. $O(n^3)$
D. $O(n^4)$ E. None of these

QUESTION 31

Which of these could replace `<*1>` in the code to the right as a valid call to `f()`?

- A. `f(a,&b)` B. `f(&a,b)` C. `f(a,*b)`
D. `f(a,b)` E. None of these

QUESTION 32

For which of the following values of `int z` will `display(z)` output at least one `'*'`?

- A. 79 B. -79 C. 85
D. 21 E. More than one of these

```
template <class itemType>
void sort(apvector<itemType> & A){
    sort1(A,0,A.length()-1);
}
```

```
template <class itemType>
void sort1(apvector<itemType> & A,
           int x, int y) {
    int z=(x+y)/2;
    if (z-x>=1) sort1(A,x,z); /**/
    if (y-z>1) sort1(A,z+1,y);
    apvector<itemType>
        B(y-x+1,A[z]);
    int i=x,j=z+1,k=0;
    while (i<=z && j<=y)
        if (A[i]<A[j]) B[k++]=A[i++];
        else B[k++]=A[j++];
    while (i<=z) B[k++]=A[i++];
    while (j<=y) B[k++]=A[j++];
    for (k=0; k<=y-x; ++k)
        A[x+k]=B[k];
}
```

```
int f(int &x, int *y) {
    // do something
}
```

```
int main() {
    int a,b;
    cout << <*1>;
}
```

```
void display(int z) {
    while (z>0) {
        if(z%4 == 0) cout << '*';
        z/=4;
    }
}
```

QUESTION 33

Which of these fills `<*1>` in the code to the right to set `r` to be the number of rows of `A`?

- A. `numrows(A)` B. `A.numrows`
 C. `num_rows(A)` D. `A.num_rows`
 E. None of these

QUESTION 34

Assume that `<*1>` is filled in correctly. What will be returned by the function call `path(A, 3, 6)` where `A` is the matrix below?

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	0	3	0	0	7	8	0	4
[1]	0	0	0	2	7	7	7	0
[2]	0	2	0	7	3	5	0	2
[3]	6	0	1	0	1	0	0	1
[4]	8	2	2	2	0	1	0	1
[5]	0	0	0	0	0	0	0	0
[6]	0	0	0	0	0	0	0	0
[7]	4	7	0	4	8	8	0	0

- A. `false` B. `true` C. `-1`
 D. `0` E. None of these

```
bool path(apmatrix<int> A,
          int i, int j) {
    int r = <*1>;
    apvector<bool> checked(r,false);
    apvector<bool> found(r,false);
    found[i]=true;
    bool progress=true;
    while(progress) {
        progress=false;
        for (int k=0; k<r; ++k)
        {
            if (found[k]) {
                if (k==j) return true;
                progress=true;
                found[k]=false;
                checked[k]=true;
                for (int z=0; z<r; ++z) {
                    if(A[k][z] &&
                        !checked[z])
                        found[z]=true;
                }
            }
        }
    }
    return false;
}
```

QUESTION 35

What is output by the function call `f("texas")` in the code to the right?

- A. `ueyat` B. `texas`
 C. `ufybt` D. `sdw`r`
 E. None of these

```
void f(apstring s) {
    static int x=0;
    for (int i=0; i<s.length(); ++i)
    {
        if (s[i]>'m') ++x;
        else --x;
        s[i]+=x;
    }
    cout << s;
}
```

QUESTION 36

What is output by the function call `f("virginia")` in the code to the right?

- A. `wisfgmg^` B. `virginia`
 C. `uhpfhmhb` D. `wisgioia`
 E. None of these

```
int main() {
    f("texas");
    f("virginia");
    return 0;
}
```

QUESTION 37

Which of these is a legal declaration of an object of type C inside a main() function of a program?

- A. C c(true) B. C c(5,3)
 C. C c() D. C c("hello")
 E. More than one of these

```
class C {
public:
    C(int x);
    C(int x, int y);

private:
    C(apstring s);
    // data omitted
};
```

QUESTION 38

What is returned by mystery(A,10) where A is the array below?

12	8	19	2	-5	6	-2	0	4	7
----	---	----	---	----	---	----	---	---	---

- A. 0 B. 6 C. 7
 D. 12 E. None of these

```
int partition(int A[], int r, int s) {
    // rearranges elements of A
    // from index r to s-1,
    // so that all elements before
    // the element at A[r] come
    // first, then the element
    // originally at A[r], then
    // all the other elements

    // returns the new index of the
    // original A[r]
}
```

QUESTION 39

What function is mystery() computing?

- A. median B. mean C. mode
 D. cosine E. None of these

```
int mystery(int A[], int length) {
    int mid=length/2;
    int part;
    int i=0, j=length;
    while ((part=partition(A,i,j))!=mid) {
        if (part<mid) i=part+1;
        else j=part-1;
    }
    return A[mid];
}
```

QUESTION 40

Which of these loops does not terminate for all integers x?

- A. while (x&=x); B. while (x/=2); C. while (!x==x);
 D. while (x-=x); E. More than one of these

AP QUICK REFERENCE

NOTE: The front and back of this page includes reference information for the AP classes. This is the same reference information that students use during the AP exam. You may use this reference sheet during the contest. You may detach this page from the test booklet.

Quick Reference for apstring

extern const int npos; // used to indicate not a position in the string

class apstring

// public member functions

// constructors/destructor

```
apstring(); // construct empty string ""
apstring(const char * s); // construct from string literal
apstring(const apstring & str); // copy constructor
~apstring(); // destructor
```

// assignment

```
const apstring & operator= (const apstring & str); // assign str
const apstring & operator= (const char * s); // assign s
const apstring & operator= (char ch); // assign ch
```

// accessors

```
int length() const; // number of chars
int find(const apstring & str) const; // index of first occurrence of str
int find(char ch) const; // index of first occurrence of ch
apstring substr(int pos, int len) const; // substring of len chars, starting at pos
const char * c_str() const; // explicit conversion to char *
```

// indexing

```
char operator[ ](int k) const; // range-checked indexing
char & operator[ ](int k); // range-checked indexing
```

// modifiers

```
const apstring & operator+= (const apstring & str); // append str
const apstring & operator+= (char ch); // append char
```

// The following free (non-member) functions operate on strings

// I/O functions

```
ostream & operator<< ( ostream & os, const apstring & str );
istream & operator>> ( istream & is, apstring & str );
istream & getline( istream & is, apstring & str );
```

// comparison operators

```
bool operator== ( const apstring & lhs, const apstring & rhs );
bool operator!= ( const apstring & lhs, const apstring & rhs );
bool operator< ( const apstring & lhs, const apstring & rhs );
bool operator<= ( const apstring & lhs, const apstring & rhs );
bool operator> ( const apstring & lhs, const apstring & rhs );
bool operator>= ( const apstring & lhs, const apstring & rhs );
```

// concatenation operator +

```
apstring operator+ ( const apstring & lhs, const apstring & rhs );
apstring operator+ ( char ch, const apstring & str );
apstring operator+ ( const apstring & str, char ch );
```

AP QUICK REFERENCE CONT.

Quick Reference for apvector and apmatrix

```
template <class itemType>
class apvector

// public member functions

// constructors/destructor
apvector(); // default constructor (size==0)
apvector(int size); // initial size of vector is size
apvector(int size, const itemType & fillValue); // all entries == fillValue
apvector(const apvector & vec); // copy constructor
~apvector(); // destructor

// assignment
const apvector & operator= (const apvector & vec);

// accessors
int length() const; // capacity of vector

// indexing
itemType & operator[ ](int index); // indexing with range checking
const itemType & operator[ ](int index) const; // indexing with range checking

// modifiers
void resize(int newSize); // change size dynamically; can result in losing values
```

```
template <class itemType>
class apmatrix

// public member functions

// constructors/destructor
apmatrix(); // default size is 0 x 0
apmatrix(int rows, int cols); // size is rows x cols
apmatrix(int rows, int cols, const itemType & fillValue); // all entries==fillValue
apmatrix(const apmatrix & mat); // copy constructor
~apmatrix( ); // destructor

// assignment
const apmatrix & operator = (const apmatrix & rhs);

// accessors
int numrows() const; // number of rows
int numcols() const; // number of columns

// indexing
const apvector<itemType> & operator[ ](int k) const; // range-checked indexing
apvector<itemType> & operator[ ](int k); // range-checked indexing

// modifiers
void resize(int newRows, int newCols); // resizes matrix to newRows x newCols
// (can result in losing values)
```

AP QUICK REFERENCE CONT.

Quick Reference for apstack and apqueue

```
template <class itemType>
class apstack

// public member functions

// constructors/destructor
apstack(); // construct empty stack
apstack(const apstack & s); // copy constructor
~apstack(); // destructor

// assignment
const apstack & operator= (const apstack & s);

// accessors
const itemType & top() const; // return top element (NO pop)
bool isEmpty() const; // return true if empty, else false
int length() const; // returns number of elements in stack

// modifiers
void push(const itemType & item); // push item onto top of stack
void pop(); // pop top element
void pop(itemType & item); // combines pop and top
void makeEmpty(); // make stack empty (no elements)

template <class itemType>
class apqueue

// public member functions

// constructors/destructor
apqueue(); // construct empty queue
apqueue(const apqueue & q); // copy constructor
~apqueue(); // destructor

// assignment
const apqueue & operator = (const apqueue & rhs);

// accessors
const itemType & front() const; // return front (no dequeue)
bool isEmpty() const; // return true if empty else false
int length() const; // return number of elements in queue

// modifiers
void enqueue(const itemType & item); // insert item (at rear)
void dequeue(); // remove first element
void dequeue(itemType & item); // combine front and dequeue
void makeEmpty(); // make queue empty
```

Computer Science Answer Key

UIL State 2003

1. C	11. C	21. E	31. A
2. C	12. B	22. C	32. A
3. D	13. A	23. B	33. E
4. C	14. E	24. A	34. B
5. C	15. B	25. A	35. A
6. D	16. D	26. B	36. E
7. D	17. D	27. B	37. B
8. D	18. A	28. B	38. C
9. B	19. B	29. D	39. A
10. B	20. B	30. E	40. A